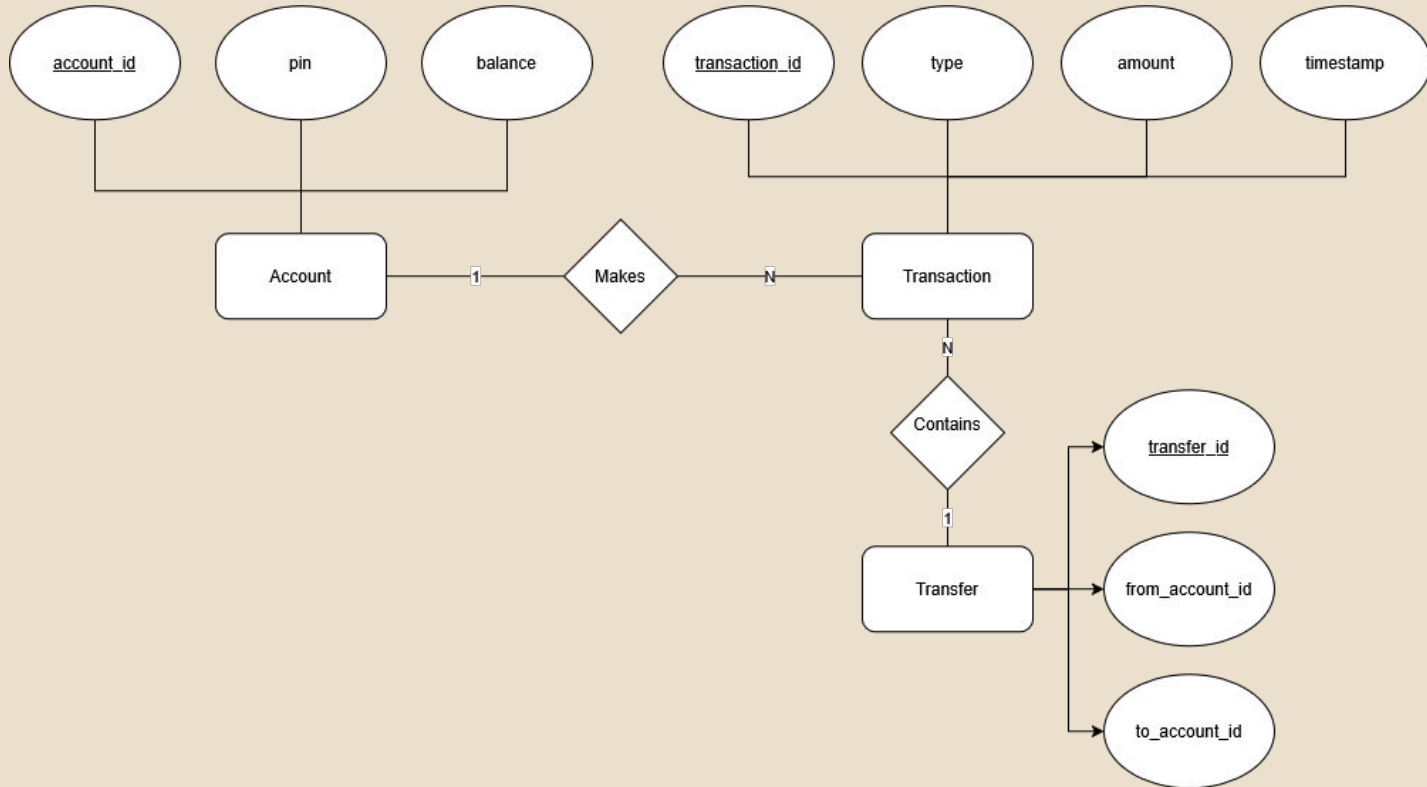

Bank of CLI Project

Canly Kong

ER Diagram



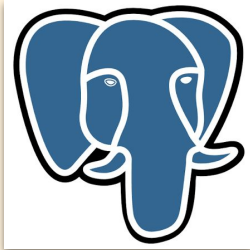
Technologies

Language



Java

Database



PostgreSQL

Build Tool



Maven

Testing



JUnit5

Version Control



Git/GitHub

Architecture

Multi-Layered Application

Presentation/UI Layer

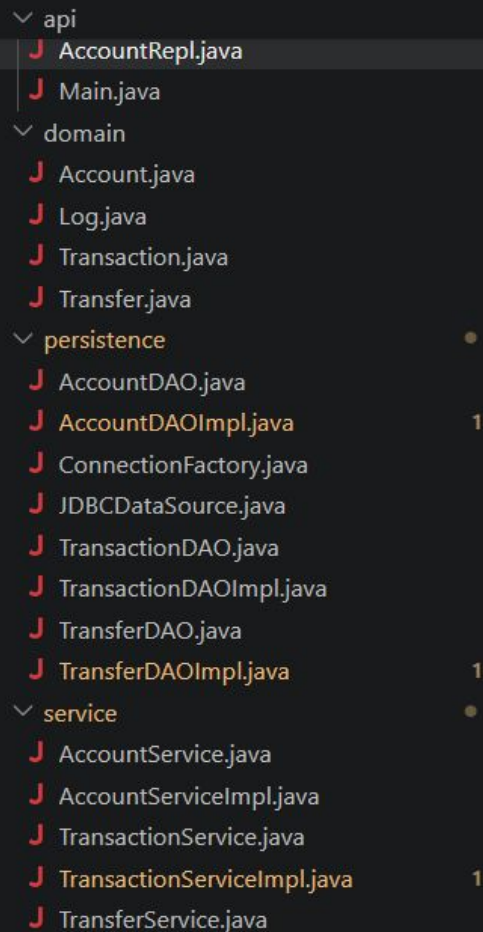
- User Interaction
- Input parsing
- Output Formatting
- Menu navigation

Service Layer

- Business concepts
- Rules/logic
- Workflow

Persistence Layer

- Interacts with Database
- Persistence
- Uses SQL



The screenshot shows a file explorer with the following structure:

- api
 - AccountRepl.java
 - Main.java
- domain
 - Account.java
 - Log.java
 - Transaction.java
 - Transfer.java
- persistence
 - AccountDAO.java
 - AccountDAOImpl.java (1)
 - ConnectionFactory.java
 - JDBCDataSource.java
 - TransactionDAO.java
 - TransactionDAOImpl.java
 - TransferDAO.java
 - TransferDAOImpl.java (1)
- service
 - AccountService.java
 - AccountServiceImpl.java
 - TransactionService.java
 - TransactionServiceImpl.java (1)
 - TransferService.java

Encapsulation

```
public class Account {  
    private int id;  
    private String pin;  
    private BigDecimal balance;  
  
    public Account(int id, String pin, BigDecimal balance) {  
        this.id = id;  
        this.pin = pin;  
        this.balance = balance;  
    }  
  
    public int getID() {  
        return id;  
    }  
  
    public String getPin() {  
        return pin;  
    }  
  
    public BigDecimal getBalance() {  
        return balance;  
    }  
  
    public void setBalance(BigDecimal amount) {  
        balance = amount;  
    }  
}
```

Polymorphism

```
// Constructor for making new Transactions
public Transaction(String type, BigDecimal amount, LocalDateTime timestamp, int account_id, Integer transfer_id) {
    this.type = type;
    this.amount = amount;
    this.timestamp = timestamp;
    this.account_id = account_id;
    this.transfer_id = transfer_id;
}

// Constructor for reading Transactions
public Transaction(int id, String type, BigDecimal amount, LocalDateTime timestamp, int account_id, Integer transfer_id, Integer from_account_id, Integer to_account_id) {
    this.id = id;
    this.type = type;
    this.amount = amount;
    this.timestamp = timestamp;
    this.account_id = account_id;
    this.transfer_id = transfer_id;
    this.from_account_id = from_account_id;
    this.to_account_id = to_account_id;
}
```

Abstraction

```
package com.bankofcli.service;

import com.bankofcli.domain.Account;
import java.math.BigDecimal;

public interface AccountService {
    void addAccount(Account account);
    Account getAccount(Account account);
    BigDecimal getBalance(Account account);
    void deposit(Account account, BigDecimal amount);
    void withdraw(Account account, BigDecimal amount);
    void transfer(Account account, BigDecimal amount, Integer transfer_to_id);
}
```

```
private void handleLoggedIn(String command) {
    switch (command) {
        case "balance" -> getBalance();
        case "deposit" -> service.deposit(this.currentAccount, readDecimal(prompt: "Deposit amount: "));
        case "withdraw" -> handleWithdraw();
        case "transfer" -> handleTransfer();
        case "history" -> transactionService.findAllTransactions(this.currentAccount.getID()).forEach(System.out::println);
        case "logout" -> this.currentAccount = null;
        case "help" -> printHelpLoggedIn();
        default -> System.out.println(x: "Unknown command");
    }
}
```

Features

Users can:

- Create and login to their accounts with an ID and PIN
- Check the current balance of their account
- Withdraw and deposit funds
- Transfer funds from their account to another existing account
- View transaction history

Logged out:

Available commands:

create - Create an account
login - Login to an existing account
help - Show this help message
exit - Exit the application

Logged in:

Available commands:

balance - Show your account balance
deposit - Deposit an amount
withdraw - Withdraw an amount
transfer - Transfer an amount
history - View transaction history
logout - Logout of your account
help - Show this help message
exit - Exit the application

Logging

Info Messages:

- What services are being used
- Records expected events
- Records successful events

```
110 void addAccount(Account account) {  
    logger.info(format: "Creating new account with ID:{}", account.getID());  
    Connection connection = JDBCDataSource.getConnection();
```

```
2026-09-23 09:57:12 INFO [main] c.b.service.AccountServiceImpl - Creating new account with ID:30
```

Error Messages:

- Records failures in services
- Records exceptions
- Records unexpected events

```
logger.warn(format: "Account ID: {} already exists", account.getID());
```

```
2026-09-18 15:56:11 WARN [main] c.b.service.AccountServiceImpl - Account with ID: 1010 already exists!
```

Testing

Positive Tests:

- Expected input
- Expected Success

```
@Test
void addDepositUpdatesBalance() {
    BigDecimal original_amount = BigDecimal.valueOf(val: 10.00);
    BigDecimal deposit_amount = BigDecimal.valueOf(val: 20.00);
    Account account = account(id: 1, pin: "1234", original_amount);
    service.addAccount(account);

    service.deposit(account, deposit_amount);

    BigDecimal new_balance = original_amount.add(deposit_amount);
    assertEquals(account.getBalance(), new_balance);
}
```

Negative Tests:

- Invalid Input
- Expected Failure

```
@Test
void addAccountAlreadyExists() {
    Account account = account(id: 1, pin: "1234", BigDecimal.ZERO);
    Account second_account = account(id: 1, pin: "1234", BigDecimal.ZERO);
    service.addAccount(account);

    IllegalArgumentException exception = assertThrows(expectedType: IllegalArgumentException.class, () -> {
        service.addAccount(second_account);
    });
    assertEquals(expected: "Account already exists", exception.getMessage());
}
```